



FACULTAD DE  
**ESTUDIOS PROFESIONALES**  
**ZONA MEDIA**



UNIVERSIDAD AUTÓNOMA DE SAN LUIS POTOSÍ

FACULTAD DE ESTUDIOS PROFESIONALES ZONA MEDIA

## Práctica 9

Contador Bidireccional 0–99

con Multiplexeo de Dos Displays

de 7 Segmentos en Raspberry Pi Pico W

**Carrera:**

Ingeniería Mecatrónica – Cuarto Semestre

**Alumno:**

Saúl Martínez Almazán

**Docente:**

Ing. Luis Jesús Padrón Padrón

**Fecha:**

3 de Marzo de 2026

# Contents

|          |                                                                                                  |          |
|----------|--------------------------------------------------------------------------------------------------|----------|
| <b>1</b> | <b>Introducción</b>                                                                              | <b>2</b> |
| 1.1      | Objetivo . . . . .                                                                               | 2        |
| 1.2      | Trabajo Previo: Concepto de Multiplexeo . . . . .                                                | 2        |
| 1.2.1    | Monitor Serial en Visual Studio Code con PlatformIO . . . . .                                    | 3        |
| <b>2</b> | <b>Desarrollo</b>                                                                                | <b>3</b> |
| 2.1      | Diseño del Circuito . . . . .                                                                    | 3        |
| 2.2      | Tabla de Patrones de Segmentos . . . . .                                                         | 4        |
| 2.3      | Proceso de Desarrollo con PlatformIO . . . . .                                                   | 4        |
| 2.4      | Errores Comunes y Soluciones . . . . .                                                           | 5        |
| <b>3</b> | <b>Preguntas de Repaso</b>                                                                       | <b>5</b> |
| 3.1      | ¿Por qué se utiliza <code>millis()</code> en lugar de <code>delay()</code> para el contador? . . | 5        |
| 3.2      | Explicación de la función <code>dibujarDigito()</code> . . . . .                                 | 6        |
| 3.3      | ¿Cómo funciona la inversión de dirección al presionar el botón? . . . .                          | 6        |
| <b>4</b> | <b>Conclusión</b>                                                                                | <b>6</b> |
|          | <b>Anexos</b>                                                                                    | <b>7</b> |

# 1 Introducción

En esta práctica se extiende el trabajo realizado en la Práctica 8 para controlar **dos displays de 7 segmentos de cátodo común** que muestran un contador decimal de dos dígitos con rango de **0 a 99**. El conteo inicia de forma ascendente al encender la placa y, al presionar un botón, la dirección se **invierte** desde el valor actual, descendiendo hasta 0 para volver a ascender, o viceversa, creando un ciclo continuo. La técnica fundamental que hace posible controlar ambos displays con un único juego de pines de segmentos es el **multiplexeo por tiempo**, investigada y aplicada durante el desarrollo de la práctica.

Adicionalmente, el código implementa comunicación serial para enviar el estado del contador al **Monitor Serial de Visual Studio Code** a través de PlatformIO, permitiendo supervisar en tiempo real el valor del contador y la dirección activa.

## 1.1 Objetivo

Comprender e implementar la técnica de multiplexeo temporal para controlar múltiples displays de 7 segmentos con un número reducido de pines GPIO, diseñar la lógica de un contador bidireccional con detección de flancos en botón y anti-rebote por software, y visualizar datos de depuración en el Monitor Serial integrado de VS Code.

## 1.2 Trabajo Previo: Concepto de Multiplexeo

El **multiplexeo** (del inglés *multiplexing*) es una técnica que permite compartir un canal o recurso físico entre múltiples señales o dispositivos de forma secuencial en el tiempo, de modo que cada uno ocupa el medio durante una fracción de tiempo muy breve.

En el contexto de displays de 7 segmentos, el **multiplexeo por tiempo** (Time-Division Multiplexing, TDM) consiste en activar un solo display a la vez mientras los demás permanecen apagados, alternando entre ellos a una frecuencia suficientemente alta (típicamente 50–200 Hz por dígito) para que el ojo humano, cuya *persistencia de la visión* retiene imágenes aproximadamente  $1/25$  de segundo, perciba ambos encendidos simultáneamente.

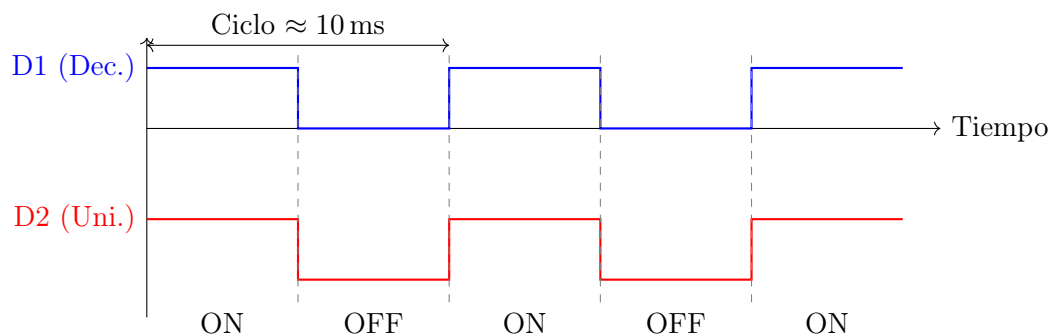


Figure 1: Diagrama de temporización del multiplexeo: cada display se activa alternadamente durante  $\approx 5$  ms, logrando una frecuencia de refresco de  $\approx 100$  Hz por dígito.

La ventaja principal es la reducción de pines GPIO: sin multiplexeo, dos displays de 7 segmentos requerirían 14 pines de segmentos; con multiplexeo basta con **7 pines compartidos** más 1 pin de selección por display, es decir,  $7 + 2 = 9$  pines en total.

### 1.2.1 Monitor Serial en Visual Studio Code con PlatformIO

Para visualizar los datos enviados por `Serial.print()` desde la Raspberry Pi Pico W, PlatformIO integra un monitor serial directamente en VS Code. Los pasos para abrirlo son:

1. Asegurarse de que `monitor_speed = 115200` esté definido en `platformio.ini` (debe coincidir con el valor en `Serial.begin(115200)`).
2. En la barra de estado inferior de VS Code, hacer clic en el ícono de enchufe (**Serial Monitor**) o usar el atajo `Ctrl+Alt+S` / `Cmd+Alt+S`.
3. Alternativamente, abrir la **PlatformIO Home** → *Devices* y hacer clic en *Monitor* junto al puerto COM/tty detectado.
4. También es posible ejecutar desde la terminal integrada: `pio device monitor -baud 115200`.

Una vez abierto, cada llamada a `Serial.println()` aparece como una línea de texto en tiempo real, permitiendo depurar el valor del contador y la dirección sin necesidad de herramientas externas.

## 2 Desarrollo

### 2.1 Diseño del Circuito

El circuito utiliza la técnica de **cátodo común multiplexado**: los siete pines de segmentos son compartidos por ambos displays; los cátodos comunes de cada display se conectan a pines GPIO independientes que actúan como selectores.

- **Pines de segmentos compartidos (A–G):** GP28, GP27, GP12, GP13, GP14, GP22 y GP26, cada uno conectado a través de una resistencia de  $220\ \Omega$  a los segmentos correspondientes de *ambos* displays en paralelo.
- **Cátodo del Display de Decenas:** GP11. Nivel LOW activa el display; nivel HIGH lo apaga.
- **Cátodo del Display de Unidades:** GP10. Mismo criterio.
- **Pulsador:** GP15, configurado con resistencia interna *pull-up* (`INPUT_PULLUP`); activo en nivel bajo.

En los displays de **cátodo común** el cátodo de todos los LEDs internos está unido a GND; para encender un segmento se lleva su pin de ánodo a nivel alto (HIGH). Por tanto, la lógica de los segmentos es **directa** (1 enciende, 0 apaga) y el selector de dígito se activa con LOW (conecta el cátodo a tierra cerrando el circuito).

El cálculo de la resistencia limitadora de segmento considera que en el peor caso (dígito 8, los 7 segmentos encendidos) cada segmento conduce individualmente:

$$R = \frac{V_{CC} - V_{LED}}{I_{seg}} = \frac{3.3 - 2.0}{0.010} = 130 \Omega \rightarrow 220 \Omega \text{ (valor comercial, mayor margen de seguridad)}$$

## 2.2 Tabla de Patrones de Segmentos

Para los dígitos del 0 al 9 en cátodo común, el bit en 1 enciende el segmento correspondiente.

Table 1: Patrones de segmentos para dígitos 0–9 (cátodo común)

| Dígito | Hex  | Binario (gfedcba) | A | B | C | D | E | F | G |
|--------|------|-------------------|---|---|---|---|---|---|---|
| 0      | 0x3F | 0011 1111         | 1 | 1 | 1 | 1 | 1 | 1 | 0 |
| 1      | 0x06 | 0000 0110         | 0 | 1 | 1 | 0 | 0 | 0 | 0 |
| 2      | 0x5B | 0101 1011         | 1 | 1 | 0 | 1 | 1 | 0 | 1 |
| 3      | 0x4F | 0100 1111         | 1 | 1 | 1 | 1 | 0 | 0 | 1 |
| 4      | 0x66 | 0110 0110         | 0 | 1 | 1 | 0 | 0 | 1 | 1 |
| 5      | 0x6D | 0110 1101         | 1 | 0 | 1 | 1 | 0 | 1 | 1 |
| 6      | 0x7D | 0111 1101         | 1 | 0 | 1 | 1 | 1 | 1 | 1 |
| 7      | 0x07 | 0000 0111         | 1 | 1 | 1 | 0 | 0 | 0 | 0 |
| 8      | 0x7F | 0111 1111         | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 9      | 0x6F | 0110 1111         | 1 | 1 | 1 | 1 | 0 | 1 | 1 |

## 2.3 Proceso de Desarrollo con PlatformIO

1. **Creación del proyecto:** Desde PlatformIO se creó la carpeta `Practica_9` seleccionando la placa *Raspberry Pi Pico W* con el framework *Arduino*.
2. **Planificación del multiplexeo:** Se determinó una pausa de 5ms por dígito, obteniendo una frecuencia de refresco de  $\approx 100$  Hz, suficiente para eliminar el parpadeo visible.
3. **Escritura del código:** Se separaron tres responsabilidades principales en `loop()`: gestión del contador con intervalos no bloqueantes (`millis()`), detección del botón con anti-rebote por software, y el ciclo de multiplexeo con la función auxiliar `dibujarDigito()`.
4. **Uso de `millis()` en lugar de `delay()`:** Se utilizó `millis()` para el intervalo de conteo de 200ms con el fin de no bloquear el ciclo de multiplexeo: si `loop()` se bloquease con `delay(200)`, los displays dejarían de actualizarse y aparecería el efecto *fantasma* o apagado visible durante el intervalo.

5. **Compilación y carga:** Se compiló con *Build* y se cargó a la Pico W en modo BOOTSEL mediante *Upload*.
6. **Verificación:** Al encender, los displays incrementan de 00 a 99; al presionar el botón, el conteo se invierte desde el valor actual; el Monitor Serial muestra cada cambio en tiempo real.

## 2.4 Errores Comunes y Soluciones

Table 2: Errores encontrados y soluciones aplicadas durante el desarrollo

| Error Encontrado                                                           | Solución Aplicada                                                                                                                                                                                                               |
|----------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Efecto “fantasma”: un dígito aparece tenuemente en el display incorrecto.  | Se apagan ambos cátodos (HIGH) <i>antes</i> de cambiar los segmentos en <code>dibujarDigito()</code> , asegurando que los nuevos patrones se establezcan con los displays apagados.                                             |
| El contador salta valores o rebota al presionar el botón.                  | Se implementó anti-rebote por software: el cambio de dirección sólo se registra si han transcurrido al menos <code>delayRebote = 50ms</code> desde el último flanco detectado, y únicamente al detectar flanco de bajada (LOW). |
| Los displays parpadean visiblemente o uno se ve más brillante que el otro. | Se verificó que la pausa por dígito fuera de exactamente 5 ms para cada display, equilibrando el ciclo de trabajo al 50% y manteniendo la frecuencia de refresco sobre 50 Hz.                                                   |
| El monitor serial no muestra datos o aparece basura.                       | Se confirmó que <code>monitor_speed = 115200</code> en <code>platformio.ini</code> coincide con <code>Serial.begin(115200)</code> en el código, y se reinició el Monitor Serial tras programar la placa.                        |

## 3 Preguntas de Repaso

### 3.1 ¿Por qué se utiliza `millis()` en lugar de `delay()` para el contador?

La función `delay(n)` detiene completamente la ejecución del programa durante  $n$  milisegundos. Si el contador usara `delay(200)`, el ciclo `loop()` quedaría bloqueado 200 ms en cada incremento; durante ese tiempo la función `dibujarDigito()` no se

llamaría, los cátodos permanecerían sin actualizar y los displays se apagarían o mostrarían un valor congelado visible.

Con `millis()` se implementa un **temporizador no bloqueante**: en cada iteración de `loop()` se comprueba si ha transcurrido el intervalo deseado sin detener el flujo del programa. Así el multiplexeo continúa ejecutándose a plena velocidad ( $\approx 100$  Hz) mientras simultáneamente se detecta el botón y se controla el tiempo del contador.

### 3.2 Explicación de la función `dibujarDigito()`

La función recibe el **número a mostrar** y el **pin del cátodo** que corresponde al display activo, y ejecuta tres pasos:

1. **Apagar ambos displays** (HIGH en ambos cátodos): desconecta temporalmente todos los LEDs para evitar que el patrón del nuevo número aparezca un instante en el display incorrecto (efecto fantasma).
2. **Configurar los segmentos**: itera sobre los 7 pines de segmento y escribe el valor correspondiente de la tabla `numeros[numero][i]`.
3. **Activar únicamente el display deseado** (LOW en el pin de cátodo indicado): cierra el circuito de ese display y enciende los segmentos configurados.

Este esquema, repetido alternadamente para decenas y unidades con 5 ms de pausa, produce la ilusión de encendido simultáneo.

### 3.3 ¿Cómo funciona la inversión de dirección al presionar el botón?

La variable `direccion` toma únicamente dos valores: `+1` (ascendente) o `-1` (descendente). En cada ciclo de conteo se suma `contador += direccion`, por lo que el signo de `direccion` determina si el valor crece o decrece.

Al detectar el flanco de bajada del botón (transición de HIGH a LOW), la instrucción `direccion *= -1` multiplica el valor actual por `-1`, invirtiendo el signo: si valía `+1` pasa a `-1`, y viceversa. Esto garantiza que el cambio ocurra desde el número exacto en pantalla sin reiniciar el contador. Los límites 0 y 99 se manejan con envolvimiento (*wrap-around*): si `contador` supera 99 vuelve a 0, y si baja de 0 sube a 99.

## 4 Conclusión

La práctica integró conceptos clave del desarrollo con microcontroladores: la técnica de **multiplexeo por tiempo** demostró cómo reducir el uso de GPIOs compartiendo pines de segmentos entre dos displays, activando cada uno alternadamente a una frecuencia imperceptible para el ojo humano. El uso de `millis()` como temporizador no bloqueante resultó fundamental para mantener simultáneamente el refresco del display, la detección

del botón y el control del intervalo de conteo sin recurrir a retardos que interrumpieran el flujo del programa.

La comunicación serial con el Monitor Serial de VS Code/PlatformIO añadió una herramienta de depuración eficaz, permitiendo verificar el valor del contador y la dirección activa en tiempo real sin instrumentación adicional.

#### Posibles mejoras:

- **Ampliar a más dígitos:** Extender el multiplexeo a 3 o 4 displays para contadores de 0 a 999 o 9 999 con pines mínimos.
- **Control de velocidad:** Agregar un segundo botón o un potenciómetro que modifique el intervalo de conteo en tiempo real.
- **Interrupciones por hardware:** Reemplazar el polling del botón por una IRQ en flanco de bajada para respuesta inmediata y menor carga al procesador.
- **Temporizador por hardware (PWM/Timer):** Usar un timer de hardware para gestionar el multiplexeo de forma completamente independiente al `loop()`, garantizando frecuencias de refresco estables.

## Anexos

### Anexo A – Código Fuente Completo (main.cpp)

```
1  #include <Arduino.h>
2
3  // --- DEFINICION DE PINES ---
4  // Segmentos A, B, C, D, E, F, G
5  const int segmentPins[] = {28, 27, 12, 13, 14, 22, 26};
6
7  // Catodos Comunes (Digitos)
8  const int pinDecenas = 11;
9  const int pinUnidades = 10;
10
11 // Boton
12 const int pinBoton = 15;
13
14 // --- MAPA DE NUMEROS (0-9) ---
15 // Configuracion para CATODO COMUN:
16 // 1 (HIGH) = Enciende el segmento
17 // 0 (LOW) = Apaga el segmento
18 // Orden: A, B, C, D, E, F, G
19 const byte numeros[10][7] = {
20     {1, 1, 1, 1, 1, 1, 0}, // 0
21     {0, 1, 1, 0, 0, 0, 0}, // 1
22     {1, 1, 0, 1, 1, 0, 1}, // 2
23     {1, 1, 1, 1, 0, 0, 1}, // 3
```



```
24 {0, 1, 1, 0, 0, 1, 1}, // 4
25 {1, 0, 1, 1, 0, 1, 1}, // 5
26 {1, 0, 1, 1, 1, 1, 1}, // 6
27 {1, 1, 1, 0, 0, 0, 0}, // 7
28 {1, 1, 1, 1, 1, 1, 1}, // 8
29 {1, 1, 1, 1, 0, 1, 1} // 9
30 };
31
32 // --- VARIABLES GLOBALES ---
33 int contador = 0;
34 int direccion = 1; // 1 = Ascendente, -1 = Descendente
35 unsigned long ultimoTiempoConteo = 0;
36 const int intervaloConteo = 200; // 200 ms entre incrementos
37
38 // Variables para el boton (anti-rebote por software)
39 int ultimoEstadoBoton = HIGH;
40 unsigned long ultimoTiempoRebote = 0;
41 const int delayRebote = 50; // 50 ms
42
43 void setup() {
44     Serial.begin(115200);
45
46     // Pines de segmentos como salida
47     for (int i = 0; i < 7; i++) {
48         pinMode(segmentPins[i], OUTPUT);
49     }
50
51     // Pines de catodos como salida
52     pinMode(pinDecenas, OUTPUT);
53     pinMode(pinUnidades, OUTPUT);
54
55     // Boton con pull-up interno
56     pinMode(pinBoton, INPUT_PULLUP);
57
58     // Apagar displays al inicio (HIGH en catodo = apagado)
59     digitalWrite(pinDecenas, HIGH);
60     digitalWrite(pinUnidades, HIGH);
61
62     Serial.println("--- Sistema Iniciado ---");
63 }
64
65 // Muestra 'numero' en el display seleccionado por 'pinDigito'
66 void dibujarDigito(int numero, int pinDigito) {
67     // 1. Apagar ambos displays para evitar efecto fantasma
68     digitalWrite(pinDecenas, HIGH);
69     digitalWrite(pinUnidades, HIGH);
70
71     // 2. Configurar segmentos
```

```
72   for (int i = 0; i < 7; i++) {
73       digitalWrite(segmentPins[i], numeros[numero][i]);
74   }
75
76   // 3. Activar unicamente el display deseado
77   digitalWrite(pinDigito, LOW);
78 }
79
80 void loop() {
81     unsigned long tiempoActual = millis();
82
83     // --- 1. LOGICA DEL CONTADOR (no bloqueante) ---
84     if (tiempoActual - ultimoTiempoConteo >= intervaloConteo) {
85         contador += direccion;
86
87         // Envolvimiento en limites 0-99
88         if (contador > 99) contador = 0;
89         if (contador < 0) contador = 99;
90
91         Serial.print("Contador: ");
92         Serial.print(contador);
93         Serial.print(" | Direccion: ");
94         Serial.println(direccion == 1 ? "ASC (+)" : "DESC (-)");
95
96         ultimoTiempoConteo = tiempoActual;
97     }
98
99     // --- 2. LOGICA DEL BOTON (deteccion de flanco + anti-
100     rebote) ---
101     int lecturaBoton = digitalRead(pinBoton);
102
103     if (lecturaBoton != ultimoEstadoBoton) {
104         if ((tiempoActual - ultimoTiempoRebote) > delayRebote) {
105             if (lecturaBoton == LOW) {
106                 direccion *= -1; // Invierte la direccion
107                 Serial.println(">> BOTON PRESIONADO: Cambiando
108                             direccion <<");
109             }
110             ultimoTiempoRebote = tiempoActual;
111         }
112         ultimoEstadoBoton = lecturaBoton;
113
114         // --- 3. MULTIPLEXEO (refresco continuo de ambos displays)
115         ---
116         int decenas = contador / 10;
117         int unidades = contador % 10;
```

```
117   dibujarDigito(decenas, pinDecenas);  
118   delay(5); // ~5 ms por dígito -> ~100 Hz de refresco  
119  
120   dibujarDigito(unidades, pinUnidades);  
121   delay(5);  
122 }
```

## Anexo B – Archivo platformio.ini

```
1 [env:rpipicow]  
2 platform = raspberrypi  
3 board     = rpicow  
4 framework = arduino  
5  
6 ; Velocidad del monitor serial (debe coincidir con Serial.  
   begin)  
7 monitor_speed = 115200  
8  
9 ; Opciones de compilacion  
10 build_flags =  
11     -DARDUINO_RASPBERRY_PI_PICO_W
```

## Anexo C – Diagrama de Conexiones

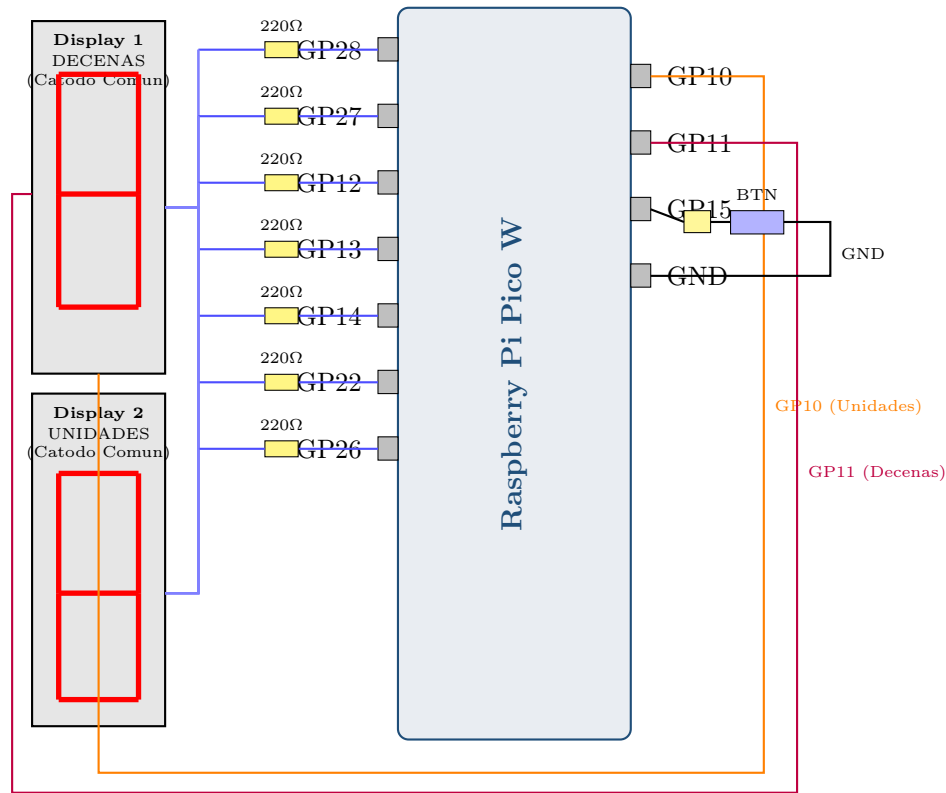


Figure 2: Diagrama esquemático del circuito: los 7 pines de segmentos se bifurcan hacia ambos displays a través de resistencias de  $220\ \Omega$ ; los pines GP10 y GP11 seleccionan el display activo; el pulsador utiliza pull-up interno con conexión a GND.

## Anexo D – Salida del Monitor Serial

El siguiente fragmento ilustra la salida típica observada en el Monitor Serial de VS Code durante la ejecución:

```

1  --- Sistema Iniciado ---
2  Contador: 1 | Direccion: ASC (+)
3  Contador: 2 | Direccion: ASC (+)
4  Contador: 3 | Direccion: ASC (+)
5  >> BOTON PRESIONADO: Cambiando direccion <<
6  Contador: 2 | Direccion: DESC (-)
7  Contador: 1 | Direccion: DESC (-)
8  Contador: 0 | Direccion: DESC (-)
9  Contador: 99 | Direccion: DESC (-)
10 >> BOTON PRESIONADO: Cambiando direccion <<
11 Contador: 0 | Direccion: ASC (+)

```

## Anexo E – Fotografía del Prototipo Armado

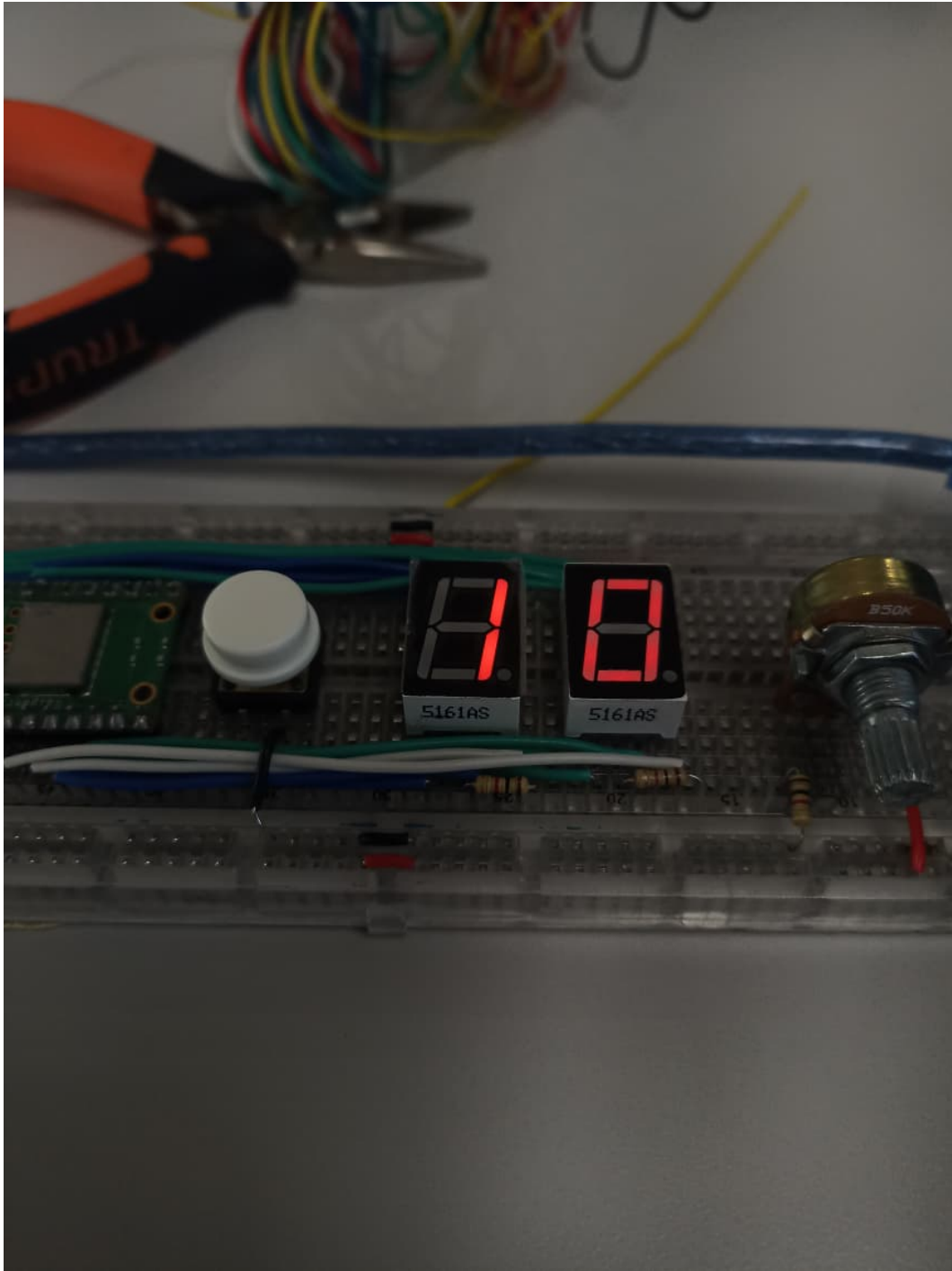


Figure 3: Fotografía del circuito armado en protoboard: Raspberry Pi Pico W con dos displays de 7 segmentos de cátodo común multiplexados y pulsador para inversión de dirección.

## References

- [1] Raspberry Pi Ltd., *Raspberry Pi Pico C/C++ SDK* (v2.0). <https://datasheets.raspberrypi.com/pico/raspberry-pi-pico-c-sdk.pdf>, 2024.
- [2] Raspberry Pi Ltd., *Raspberry Pi Pico W Datasheet*. <https://datasheets.raspberrypi.com/picow/pico-w-datasheet.pdf>, 2024.
- [3] Raspberry Pi Ltd., *RP2040 Datasheet*. <https://datasheets.raspberrypi.com/rp2040/rp2040-datasheet.pdf>, 2024.
- [4] PlatformIO Labs, *PlatformIO Documentation*. <https://docs.platformio.org>, 2024.
- [5] Texas Instruments, *Seven-Segment Display Fundamentals*. <https://www.ti.com/lit/an/slyt079/slyt079.pdf>, 2012.
- [6] Maxim Integrated, *Multiplexing Seven-Segment LED Displays*. <https://www.maximintegrated.com/en/design/technical-documents/tutorials/3/3216.html>, 2015.